



Product Requirements Document — CoalGov

AI-Based Smart Governance & Compliance Monitoring Platform for Coal Mines

SIH 2026 · PS 6024 · Ministry of Coal · Coal India Limited

Field	Value
Document owner	Team [name] · P1 (Domain/Product lead)
Status	Draft v1.0
Category / Theme	Software · Smart Automation
Target release	Hackathon MVP (14-day build) → pilot-ready framework
Last updated	[date]

1. Overview

1.1 Problem

A single Indian coal mine answers to five separate regulators — DGMS (safety), MoEFCC (environment), CPCB (pollution), Ministry of Coal (production), and Labour (contractors). Each issues rules as unstructured PDF documents on its own portal, with no shared mine identifier. Meanwhile the *proof* of compliance is recorded on bound paper registers in the field, where there is no network, and entered into systems days later. Rules and evidence never connect. Across 310 working mines this produces the exact failures the problem statement names: data inconsistency, duplication, compliance gaps, weak field monitoring, and no real-time insight.

1.2 Product vision

CoalGov is the **obligation layer** between the regulator and the mine. It ingests regulatory documents and compiles them into a live, owned, trackable task list; captures verified field evidence offline; detects compliance risk with explainable AI; and generates statutory reports automatically — delivered as an indigenous, coal-specific platform spanning web and mobile, scalable from one mine to the entire national coal sector.

1.3 What makes it different

CoalGov does not store the rulebook — it **compiles** it. Where existing systems watch from outside (CMSMS satellites), collect paperwork (PARIVESH), or run the business (SAP ERP), none turn regulations into tracked

obligations connected to verified field proof. This is the gap CoalGov fills.

1.4 Goals and non-goals

Goals

Convert regulatory PDFs into structured, owned, scheduled compliance obligations
Provide offline, geo-verified field capture with anti-spoofing trust scoring
Track the full inspection → observation → corrective action → closure loop
Cover the governance half: contractor management, grievances, attendance, digital approvals
Surface explainable compliance-failure risk and recurring violations across mines
Generate statutory reports from live data
Serve three audiences (mine, corporate, regulator) with role-scoped access

Non-goals (explicit, defensible scope boundaries)

- ✗ Not predicting fatalities or accidents (data too sparse; we predict compliance risk)
- ✗ Not a full permissioned blockchain (we use a hash-chained ledger)
- ✗ Not replacing CIL's SAP ERP (we are the compliance layer it lacks)
- ✗ Not IoT sensor hardware (this is a Software-category solution)
- ✗ Not eliminating statutory paper overnight (we run digital-first in parallel)

2. Users & personas

Persona	Role in system	Primary need	Key screens
Mine Manager / Safety Officer	MINE_OFFICIAL	"What's due from me, and am I personally exposed?"	Mine dashboard, tasks, CAPA
Sirdar / Overman / Inspector	FIELD_STAFF	Log inspections & hazards offline, fast	Mobile app only
Contractor Supervisor	CONTRACTOR	Prove workers are licensed and trained	Contractor module
Worker	WORKER	Report a hazard/grievance safely, in own language	Voice grievance
Area GM / Subsidiary Director / CIL HQ	CORPORATE	Rank mines by risk, portfolio view	Corporate dashboard
DGMS / MoEFCC official	REGULATOR	Where to inspect next; cross-mine compliance	Regulator dashboard

Scope hierarchy: every user attaches to one node — mine → area → subsidiary → corporate — and sees that node and everything below it, never above or sideways.

3. Scope by release

3.1 MVP (hackathon demo) — must ship

- | # | Capability |
|-----|---|
| M1 | Document ingestion + OCR (real EC letters, DGMS circulars) |
| M2 | Obligation extraction with orphan + conflict detection |
| M3 | Auto-generated compliance calendar with owners and SLAs |
| M4 | Offline-first mobile app (inspections, observations, incidents) |
| M5 | Inspection → CAPA workflow with escalation |
| M6 | Contractor management with certificate-expiry auto-flags |
| M7 | Grievance ticketing (incl. anonymous) |
| M8 | Alerts, reminders, escalation engine |
| M9 | Statutory report generation |
| M10 | Explainable risk engine (XGBoost + SHAP) |
| M11 | Multi-tenant RBAC (mine → corporate scoping) |
| M12 | Three dashboards (mine / corporate / regulator) |

3.2 Should-have — build if time

Evidence trust scoring · voice-first multilingual grievance (Bhashini) · geofenced attendance · hash-chained audit ledger

3.3 Stretch — architecture + mock only

Satellite verification (Sentinel-2 NDVI) · RAG compliance chatbot · production reporting · SAP/ERP integration adapter

4. Functional requirements

FR-1 • Document ingestion & obligation extraction ★

FR-1.1 Users can upload PDF/scanned regulatory documents; the system stores them and extracts text (born-digital via parser, scanned via OCR, Devanagari + English).

FR-1.2 The system segments text into clauses and extracts each as a structured obligation: `[clause_ref, obligation_text, summary, domain, frequency, responsible_role, evidence_type, mine_types, confidence]`.

FR-1.3 If a document does not state a responsible role, the obligation is flagged as an **orphan** — the system must not guess an owner.

FR-1.4 The system detects **conflicts** between obligations (contradiction, duplicate, or one-stricter) across documents and flags them for review.

FR-1.5 Each obligation retains provenance (source document + page + verbatim clause text).

Acceptance: uploading a real PARIVESH EC letter yields ≥ 1 correctly structured obligation per genuine condition, with at least one orphan and one conflict surfaced on the demo document.

FR-2 • Compliance calendar

FR-2.1 Each obligation generates recurring tasks per its frequency and due rule, 12 months ahead.

FR-2.2 Tasks auto-assign by matching `[responsible_role]` to a user's designation within the same org.

FR-2.3 Tasks carry status: pending / in_progress / submitted / verified / overdue.

FR-2.4 A nightly job flips overdue tasks and triggers notifications.

Acceptance: a half-yearly obligation produces two dated tasks per year with an owner and SLA.

FR-3 • Field capture (mobile)

FR-3.1 Field staff log inspections, observations, attendance, and incidents on mobile, fully offline.

FR-3.2 Each record captures photo, timestamp, and location — GPS for opencast, structured district code (optionally QR/NFC) for underground.

FR-3.3 Records queue locally and sync on reconnect, keyed by a device-generated `[client_uuid]` so re-sends never duplicate.

FR-3.4 The mobile UI loads the correct checklist and hazard set by `[mine_type]`.

Acceptance: with the device in airplane mode, an observation can be fully logged, and it appears on the web dashboard after reconnection with no duplicate.

FR-4 • Evidence trust scoring

FR-4.1 Each field submission receives a trust score from: mock-GPS detection, lease-boundary geofence, EXIF integrity, perceptual-hash duplicate check, and capture-vs-sync drift.

FR-4.2 The score and its component flags are visible on the evidence.

Acceptance: a gallery-sourced or out-of-geofence image scores measurably lower than a genuine in-field capture.

FR-5 · Inspection → CAPA workflow

FR-5.1 A high/critical observation auto-creates a corrective action with owner and due date.

FR-5.2 CAPAs follow a state machine: open → assigned → in_progress → closed → verified; overdue CAPAs escalate up the hierarchy.

FR-5.3 Closing a CAPA requires attached evidence.

Acceptance: an overdue CAPA visibly escalates to the next authority level automatically.

FR-6 · Contractor management

FR-6.1 Contractors and their workers are recorded with licence, VTC, and PME validity.

FR-6.2 The system auto-flags expiring/expired certificates and, at gate check-in, blocks an un-inducted or expired worker with a stated reason.

Acceptance: a worker with an expired VTC returns a "blocked" gate decision.

FR-7 · Grievance handling

FR-7.1 Workers file grievances by text or voice, optionally anonymous, in an Indian language.

FR-7.2 Voice is transcribed and translated; the ticket is auto-categorised and SLA-timed.

FR-7.3 Complainants can track status by ticket number; breaches escalate.

Acceptance: a voice grievance in Hindi becomes a categorised, SLA-timed, trackable ticket.

FR-8 · Alerts & escalation

FR-8.1 The system issues pre-deadline reminders, overdue alerts, and multi-level escalations.

FR-8.2 Digital approval requests route to the correct authority.

Acceptance: a task due in 3 days generates a reminder to its owner.

FR-9 · Risk & analytics

FR-9.1 The system scores compliance-failure risk per mine/contractor and ranks them.

FR-9.2 Every score is returned with its top contributing factors (SHAP) — never a bare number.

FR-9.3 The system detects violations recurring across multiple mines.

Acceptance: the highest-risk mine displays the specific factors that produced its score.

FR-10 · Statutory reports

FR-10.1 The system generates a statutory return (e.g. six-monthly EC compliance report) from live obligation/evidence data, in the required layout, as a downloadable PDF.

Acceptance: one click produces a condition-by-condition PDF report for a selected period.

FR-11 · Dashboards & access control

FR-11.1 Three role-scoped dashboards: mine ("due from me"), corporate ("mines by risk"), regulator ("where to inspect").

FR-11.2 RBAC enforces the scope hierarchy at the data layer; a user cannot see another branch's data.

Acceptance: logging in as three roles yields three different, correctly scoped views.

FR-12 · Audit trail

FR-12.1 Every critical action writes to an append-only, hash-chained ledger.

FR-12.2 A verification routine walks the chain and detects any altered record.

Acceptance: altering a stored record causes verification to fail at that record.

5. Non-functional requirements

Category	Requirement
Offline	Mobile app fully functional with zero connectivity; sync is idempotent
Scalability	Multi-tenant; architecture supports 310 mines across 8 subsidiaries without redesign
Sovereignty	Deployable fully on-premise / NIC MeghRaj; self-hostable LLM option; no dependency on foreign cloud for government data
Localisation	Multilingual field and grievance interfaces (Hindi, Bengali, Odia, Telugu) via Bhashini
Security	RBAC + row-level scoping; JWT auth; encrypted transport; append-only audit
Privacy	DPDP Act 2023 compliant — consent, purpose limitation, data minimisation for worker/biometric data
Explainability	No AI output presented without its reasoning (SHAP factors, extraction provenance, trust flags)
Extensibility	Regulation-agnostic obligation model — new rulebook = new config, not new code (extends to MMR 1961 metal mines)
Performance	Dashboard queries indexed on <code>(org_id, status, due_date)</code> ; document extraction async

6. Mine-type handling

The platform core is mine-type agnostic; content is scoped by `orgs.mine_type` (`underground` | `opencast` | `mixed`) and `applies_to[]` arrays on hazards and checklists.

Aspect	Underground	Opencast
Location capture	District/level code, optional QR/NFC (GPS fails underground)	GPS + lease geofence
Hazard set	roof/strata, gas, ventilation, inundation, fire	bench/highwall, dump slope, haul road, HEMM, blasting, dust
Satellite verification	Not applicable (except subsidence)	Applicable (greenbelt, dumps, reclamation)
Map view	Schematic district plan	Satellite + lease polygon

Mixed mines receive both rule sets. Adding a third profile (metalliferous, MMR 1961) requires only a new configuration.

7. Data model (core entities)

`[orgs]` (hierarchy, mine_code, mine_type, lease_boundary) · `[users]` (role, org_id, designation, is_statutory) · `[source_documents]` · `[obligations]` (typed, is_orphan, embedding) · `[obligation_conflicts]` · `[compliance_tasks]` · `[evidence]` (trust_score, phash) · `[observations]` (client_uuid, location/district) · `[capas]` (state, escalation_level) · `[contractors]` / `[contractor_workers]` (validity) · `[attendance]` (gate_decision) · `[grievances]` (ticket_no, lang, is_anonymous, sla_due) · `[risk_scores]` (factors) · `[notifications]` · `[audit_log]` (prev_hash, curr_hash).

Every business table carries `[org_id]`; one middleware injects the scope filter.

8. Key API surface


POST /documents · POST /documents/{id}/extract · GET /documents/{id}/obligations GET /obligations?orphan=true · GET /obligations/conflicts · POST /obligations/{id}/assign POST /obligations/{id}/generate-tasks GET /tasks?status=&assigned_to=me · GET /tasks/calendar · POST /tasks/{id}/submit POST /sync/push · GET /sync/pull?since= · POST /observations · POST /evidence POST /capas · PATCH /capas/{id} · POST /capas/{id}/escalate GET /contractors/expiring?days=30 · POST /attendance/checkin POST /grievances · GET /grievances/track/{ticket_no} GET /risk/mines · GET /analytics/recurring · POST /reports/generate GET /audit · POST /audit/verify

All endpoints auto-scoped by the caller's `org_id`.

9. Success metrics

9.1 Demo success (hackathon)

- ☐ Real PARIVESH EC letter ingests → obligations appear → orphan + conflict flagged
- ☐ Mobile observation logged in airplane mode → syncs → appears on dashboard
- ☐ Trust score visible on evidence
- ☐ CAPA auto-escalates when overdue
- ☐ Contractor with expiring VTC auto-flagged / blocked at gate
- ☐ Voice grievance (Hindi) → categorised, SLA-timed ticket
- ☐ Risk ranking shows factors, not just a score
- ☐ Six-monthly EC report generated as PDF in one click
- ☐ Audit verify: green → tamper a row → red
- ☐ Three roles → three correctly scoped views

9.2 Product outcome metrics (pilot framing)

Metric	Baseline today	Target
Time to build a compliance obligation list from an EC letter manual, per cycle		automated, minutes
Duplicate data entry per event	3× (register → MIS → return)	1×
Corrective actions with tracked closure	largely untracked	100% owned + SLA'd
Time to generate a statutory return	days	one click
Cross-mine recurring-violation visibility	none	automatic
Field evidence with verifiable authenticity	none	trust-scored

10. Assumptions, dependencies, risks

Assumptions

Public PARIVESH EC letters and DGMS circulars are representative training/demo corpora
Demo runs on synthetic operational data clearly labelled as such

Dependencies

Risks & mitigations

Risk	Mitigation
Live LLM call fails during demo	Cache extraction results for the 3 demo documents behind a flag
No access to CIL internal data	Public corpora + labelled synthetic ops data
Accident data too sparse for prediction	Predict compliance failure instead; state this openly
SAP integration undemonstrable	Ship the interface contract, not a fake connection
Offline sync conflicts	Idempotent <code>client_uuid</code> + server-side reconciliation
Worker data privacy	DPDP-compliant consent + minimisation

11. Release plan (14-day build)

Phase	Days	Exit gate
0 · Ground truth	0–2	Real corpus gathered, schema + obligation contract fixed, demo script written
1 · Skeleton	1–3	All clients log in and read one real endpoint
2 · Core spine	4–8	Document → obligations → tasks → offline observation → sync → CAPA → dashboard
3 · Differentiators	9–11	Trust scoring, conflict detection, voice grievance, audit chain live
4 · Harden	12–14	Feature freeze, demo rehearsed 5×, backup video recorded

Cut rule: if the core spine slips past day 8, drop stretch items immediately to protect it.

12. Open questions

Which specific CIL subsidiary/mine to theme the demo around (affects which real EC letter to feature)?

Native Flutter vs PWA for mobile, given team skills?

Which four languages to prioritise for the grievance channel in the demo?

Appendix references: PS 6024 text · CMR 2017 · Mines Act 1952 · PARIVESH · DGMS · CMSMS/Khanan Prahari · DPDP Act 2023 · RegNLP research · project Master Document, Execution Plan, and Department Workflows.